Tracing Through Langfuse

Tracing with Langfuse provides a transparent window into how your LLM applications actually perform in production. It moves beyond simple "input-output" logging to capture the entire lifecycle of a request.

---

1. How to Implement Tracing

Langfuse uses a nested structure of **Traces** and **Spans** to map out your application's logic. You can integrate it in three primary ways:

- **SDKs (Python/JS):** The most common method. You wrap your functions with decorators or manual wrappers.
- **Integrations:** If you use frameworks like **LangChain**, **LlamaIndex**, or **OpenAI SDK**, Langfuse offers one-line integrations that automatically capture all internal steps.
- **API:** For environments where you cannot install an SDK, you can send trace data directly via a REST API.

---

2. Key Elements to Trace

To get a full picture of your system, you should trace the following components:

Element	Description
Prompts	The exact version of the prompt used, including the system message and variables.
Retrieval (RAG)	The specific documents or chunks fetched from your vector database (and their relevance scores).
Model Metadata	The model name (e.g., GPT-4o, Claude 3.5), temperature, and top-p settings.
Token Usage	Precise counts for input, output, and total tokens to monitor costs.
Latency	Timestamps for the start and end of every step to find bottlenecks.
JSON Outputs	Structured data returned by the model to ensure it matches your expected schema.
User Feedback	"Thumbs up/down" or corrections provided by the end-user, linked directly to the specific trace.

---

3. How it Helps with Observability

Observability isn't just about seeing that a request failed; it’s about understanding *why*. Langfuse improves this through:

Root Cause Analysis

If a user receives a poor answer, you can look at the trace to see if the error happened because the **Retrieval** step pulled the wrong documents, the **Prompt** was ambiguous, or the **Model** hallucinated despite having the right data.

Cost & Performance Monitoring

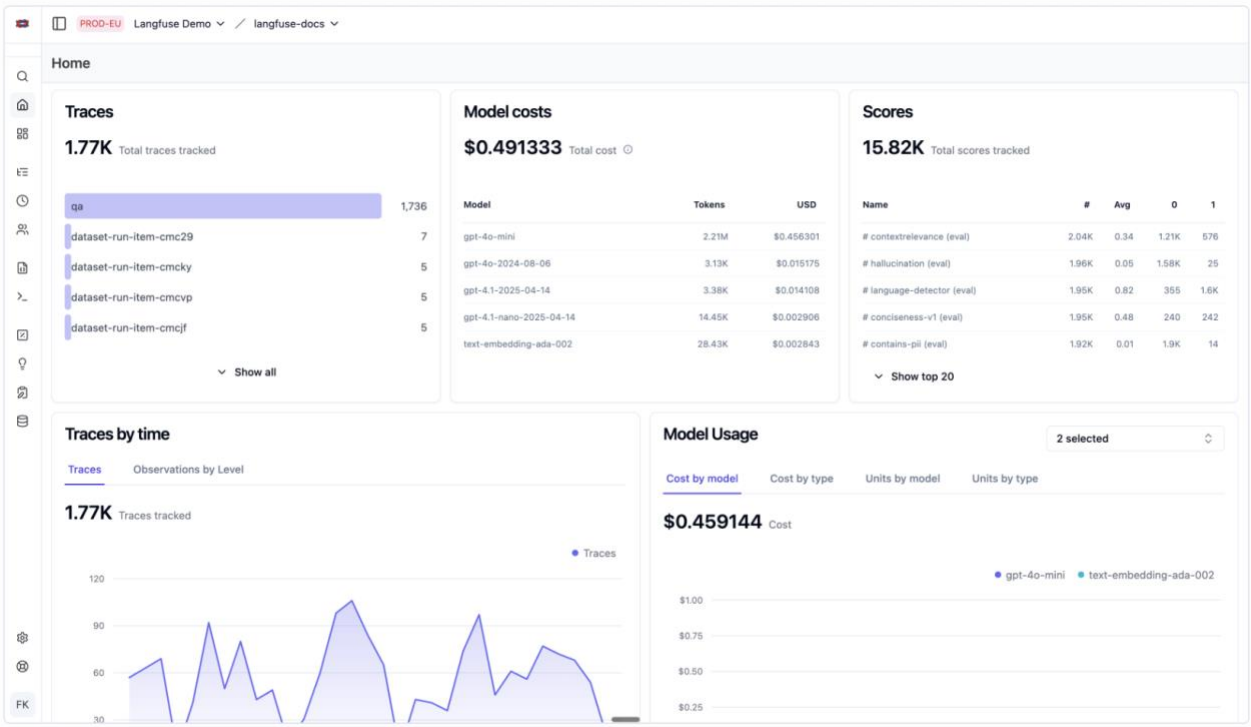
You can visualize which features are costing the most money and which steps in your "Agentic" workflow are taking the longest. This is essential for optimizing $latency$ and $COGS$ (Cost of Goods Sold).

Evaluation (Evals)

Langfuse allows you to run "evaluators" (either manual or AI-automated) on your traces. This helps you score the quality of your outputs at scale, rather than just guessing.

Dataset Generation

You can export high-quality traces to create "Golden Datasets." These are used for fine-tuning smaller models or for regression testing to ensure new prompt changes don't break existing functionality.



Eamples:

Steps:

1. Self-host Langfuse on GitHub Codespace

1. Start a new GitHub Codespace, then change its Machine Type to 4-core, and restart the Codespace.
2. Requirements: *git*, *docker*, and *docker compose*
3. Clone Langfuse Repository

   `git clone https://github.com/langfuse/langfuse.git`
4. Create a `.env` file and add your OpenAI API Key

   `OPENAI_API_KEY=ADD-YOUR-KEY-HERE`
5. Load the Environment variables (eg. OPENAI_API_KEY)

   `source .env`
6. Start the application

   `cd langfuse`

   `docker compose up`

   Watch the docker containers being started and the logs flowing in. After about 2-3 minutes, the ***langfuse-web-1*** container should log **"Ready"**.
7. Open http://localhost:3000 in your browser to access the Langfuse UI.

   (For GitHub Codespace it will be the ***Forwarded Address***. Make sure to ***make the Port Public***.)

   Next, Sign Up in Langfuse. Once Signed Up, it will take you to the localhost URL, at this point visit the Forwarded Address of GitHub Codespaces again to access Langfuse.

   Create a New Organization in Langfuse.

   Create a New Project.

   Once the Project is ready, go to *Settings (on the Left Panel) >> API Keys >> Create new API keys* >> Provide a name to the Key >> Copy the Secret Key, Public Key, Base URL and paste in the code cell below.
8. To Stop the application later

   `docker compose down`

2. OpenAI SDK (Python)

```
!pip -q install langfuse
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 462.2/462.2 kB 5.7 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 66.5/66.5 kB 2.9 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.0/88.0 kB 2.6 MB/s eta 0:00:00
```

```
LANGFUSE_SECRET_KEY="sk-lf-c404c58a-6efe-495a-bb53-b6b6a74a62f5"
LANGFUSE_PUBLIC_KEY="pk-lf-3a6cce38-0731-4c06-b13c-c74c01559caf"
LANGFUSE_BASE_URL="https://turbo-orbit-v694rgj6j7rqfpwx9-3000.app.github.dev"
```

```
# Get the below key values from your Langfuse Account

import os

os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-c404c58a-6efe-495a-bb53-b6b6a74a62f5"    # <--- change as per your Secret key

os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-3a6cce38-0731-4c06-b13c-c74c01559caf"    # <--- change as per your Public key

os.environ["LANGFUSE_BASE_URL"] = "https://turbo-orbit-v694rgj6j7rqfpwx9-3000.app.github.dev"    # <--- change as per your Base URL
```

```
from langfuse.openai import openai as langfuse_openai
```

```
# Make an API Call to OpenAI Model

response = langfuse_openai.chat.completions.create(
    name="test-chat",
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a very accurate calculator. You output only the result of the calculation."},
        {"role": "user", "content": "2 + 2 = "}],
    #metadata={"someMetadataKey": "someValue"},
    )
```

```
response
```

```
ChatCompletion(id='chatcmpl-DIZWoSkY9gH64uc6IP643W4DgK7G5', choices=[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='4', refusal=None, role='assistant', annotations=[], audio=None, function_call=None, tool_calls=None))],
created=1773318846, model='gpt-4o-mini-2024-07-18', object='chat.completion', service_tier='default', system_fingerprint='fp_e609550549',
usage=CompletionUsage(completion_tokens=1, prompt_tokens=33, total_tokens=34,
completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0),
prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

Example2:

Implement the Tracing using Langfuse for either a RAG or an Agentic application.

Take the screenshots of the Languse UI showing the Traces being logged. Save the screenshots in a document.

Submit the document and code files on LMS as submission.

```
# The 'langfuse' package name is lowercase for the install
!pip install -qU langfuse langchain-openai
```

```
# For Langfuse SDK v3.x and v4.x
from langfuse.langchain import CallbackHandler
```

```
!pip install -qU langfuse langchain langchain-openai langchain-community chromadb
```

Start coding or generate with AI.

✅ Dependencies loaded successfully!

```python
import os
from langfuse import Langfuse  # 1. Import the main client
from langfuse.langchain import CallbackHandler
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# --- 1. SETUP ---
# Ensure these environment variables are set
# Replace these with your actual keys
os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-3a6cce38-0731-4c06-b13c-c74c01559caf"
os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-c404c58a-6efe-495a-bb53-b6b6a74a62f5"
os.environ["LANGFUSE_HOST"] = "https://turbo-orbit-v694rgj6j7rqfpwx9-3000.app.github.dev"
import os
from google.colab import userdata

os.environ['OPENAI_API_KEY'] = userdata.get('OPENAI_API_KEY')
langfuse = Langfuse() # 2. Initialize the client to manage flushing
```

```python
# --- 2. RAG PIPELINE ---
langfuse_handler = CallbackHandler()

docs = ["Gemini is a multimodal AI model.", "Langfuse provides observability for LLM apps."]
vectorstore = Chroma.from_texts(docs, embedding=OpenAIEmbeddings())
retriever = vectorstore.as_retriever()

prompt = ChatPromptTemplate.from_template("Answer the question: {question} using this info: {context}")
model = ChatOpenAI(model="gpt-4o-mini")

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
```

```python
# --- 3. EXECUTION ---
question = "What is Langfuse?"

response = rag_chain.invoke(
    question,
    config={
        "callbacks": [langfuse_handler],
        "metadata": {"name": "simple-rag-demo", "user_id": "colab-user"}
    }
)

# 3. Use the client to flush, not the handler
langfuse.flush()

print(f"✅ Response: {response}")
```

```
✅ Response: Langfuse is a platform that provides observability for applications built using large language models (LLMs).
```

Report in langfuse dashboard:

See below

Home 1d Past 1 day Env default Filters Request Chart

Traces

3 Total traces tracked

Model costs

$0.000057 Total cost

Model	Tokens	USD
gpt-4o-mini-2024-07-18	256	$0.000057

Scores

0 Total scores tracked

No data

Traces by time

Traces Observations by Level

3 Traces tracked

Model Usage

All models

Cost by model Cost by type Usage by model Usage by type

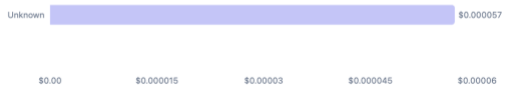
$0.000057 Cost

Home 1d Past 1 day Env default Filters Request Chart

User consumption

Token cost Count of Traces

$0.000057 Total cost



Scores

Moving average per score

No data

Trace latency percentiles

Trace Name	p50	p90	p95 ▼	p99
RunnableSequence	2.12s	2.17s	2.18s	2.19s
test-chat	1.24s	1.24s	1.24s	1.24s

Generation latency percentiles

Generation Name	p50	p90	p95 ▼	p99
RunnableSequence	2.12s	2.17s	2.18s	2.19s
ChatOpenAI	1.32s	1.35s	1.36s	1.36s
test-chat	1.24s	1.24s	1.24s	1.24s
RunnableParallel...	0.80s	0.81s	0.82s	0.82s
VectorStoreRetri...	0.80s	0.81s	0.81s	0.81s

Show all

Observation latency percentiles

Observation	p50	p90	p95 ▼	p99
CHAIN RunnableSequence	2.12s	2.17s	2.18s	2.19s
CHAIN RunnableParallel...	0.80s	0.81s	0.82s	0.82s
RETRIEVER VectorStoreRetri...	0.80s	0.81s	0.81s	0.81s
CHAIN ChatPromptTempla...	0.00s	0.00s	0.00s	0.00s
CHAIN RunnablePassthro...	0.00s	0.00s	0.00s	0.00s

Show all

Model latencies

Latencies (seconds) per LLM generation

All models

50th Percentile 75th Percentile 90th Percentile 95th Percentile 99th Percentile